Université de Béchar
Faculté des sciences et technologies
Département de Sciences Exactes
Deuxième Année LMD-MIAS
Module : Algorithmes et structures de données

Fiche supplémentaire de TD n° 01
Complexité

Exercice1 : Obtenez la complexité temporelle du programme suivant :

```
def whirlpool(m):
   if m <= 0:
      print("eddy!")
      return
   else:
      whirlpool(m - 1)
      whirlpool(m - 1)
      print("whirl")
whirlpool(3)  # Exemple d'appel de la fonction avec m = 3
```

Correction :
Tout d'abord, nous obtenons la relation de récurrence pour la complexité temporelle de la procédure "whirlpool". Soit T(m) la complexité temporelle de la procédure. La relation de récurrence est formulée comme suit :
T(m) = a, si m ≤ 0
= 2T(m - 1) + b, si m > 0.
Ici, 2T(m - 1) représente la complexité temporelle totale des deux appels à "whirlpool" (m - 1). a et b indiquent les complexités temporelles constantes pour exécuter le reste des instructions lorsque m ≤ 0 et m > 0, respectivement.
La résolution de la relation de récurrence conduit aux étapes suivantes :
T(m) = 2T(m - 1) + b ……(étape 1)
= 2(2T(m - 2) + b) + b
= 2^2 T(m - 2) + b(1 + 2).....(étape 2)
= 2^2 (2T(m - 3) + b) + b(1 + 2^2)
= 2^3 (T(m - 3) + b(1 + 2 + 2^2).....(étape 3)

En général, à l'étape i, nous avons :
T(m) = 2^i T(m - i) + b(1 + 2 + 2^2 +….2^(i-1))……(étape i)
Lorsque i = m,
T(m) = 2^m T(0) + b(1 + 2 + 2^2 +….2^(m-1))
= a·2^m + b(2^m - 1)
= k·2^m + l où k, l sont des constantes positives
= O(2^m)

La complexité temporelle de la procédure "whirlpool" est donc O(2^m).

Exercice2 : Combien de fois la ligne 3 est atteinte et traitée pendant l'exécution dans le fragment de programme suivant est ___.

a) $\frac{4n^2-2n}{2}$ b) $\frac{i^2-i}{2}$ c) $\frac{(i^2-3i)}{2}$ d) $\frac{(4n^2-6n)}{2}$

```
1.  i = 2 * n
2.  for j in range(1, i + 1):
3.   for k in range(3, j + 1):
4.   m = m + 1
```

Correction :
Le nombre de fois la ligne 3 est atteinte et traitée pendant l'exécution est donné par :

$$\sum_{j=1}^{i}(j-3+1)+1=\sum_{j=1}^{2n}(j-1)=\frac{4n^2-2n}{2}$$

Donc le choix n° 01 est la solution de cette question.

Exercice 3:
Trouver la complexité temporelle du fragment de programme suivant :

```
1. for i in range(20, 30):
2.   for j in range(1, n + 1):
3.     am = am + 1
```

Correction : la complexité temporelle du fragment de programme est :

N° de Ligne	Complexité	Explication
1	12	30-20=11 et +1 du test
2	$\sum_{j=20}^{30}(n+1)+1=11(N+1)$	
3	$\sum_{i=20}^{30}\sum_{j=1}^{n}1=11n$	

La complexité du programme est la somme des complexités de ces instructions
11(n+1)+ 11n=22n+22

Et la complexité temporelle est donc en O(n).

Exercice 4: Indiquez l'affirmation suivante qu'elle soit vraie ou fausse.

$f(n) = 30n^2 2^n + 6n2^n + 8n^2 = O(2^n)$

Correction : Fausse

Pour $f(n) = O(2^n)$, il est essentiel que : $f(n) \leq c.2^n$

C'est-à-dire : $\frac{30n^2 2^n + 6n2^n + 8n^2}{2^n} \prec c$!! qui est fausse

Exercice5 : Utilisez la notation **Grand O** pour décrire la complexité temporelle de la fonction suivante qui détermine si une année donnée est bissextile ou non:

```
def EstBissextile (annee):
   return (annee % 100 == 0) and (annee % 400 == 0) or (annee % 4 == 0)
```

Correction : Ceci est en O(1).
Nous pouvons considérer N comme l'année passée en paramètre de la fonction.
Peu importe l'année, l'algorithme ne varie pas dans le nombre d'étapes qu'il nécessite.

Exercice6 : Utilisez la notation **Grand O** pour décrire la complexité temporelle de la fonction suivante qui additionne tous les nombres d'un tableau donné :

```
def arraySum(array):
    sum = 0
    for i in range(len(array)):
        sum += array[i]
    return sum
```

Correction : C'est en O(N). Pour N éléments dans le tableau, la boucle s'exécutera N fois.

Exercice7 : La fonction suivante est basée sur l'analogie ancestrale utilisée pour décrire le pouvoir de la croissance des intérêts composés :

Imaginez que vous ayez un échiquier et que vous placiez un seul grain de riz sur une case. Sur la deuxième case, vous placez 2 grains de riz, car c'est le double de la quantité de riz sur la case précédente. Sur la troisième case, vous placez 4 grains. Sur la quatrième case, vous placez 8 grains, et sur la cinquième case, vous placez 16 grains, et ainsi de suite. La fonction suivante calcule sur quelle case vous devrez placer un certain nombre de grains de riz. Par exemple, pour 16 grains, la fonction renverra 5, car vous placerez les 16 grains sur la cinquième case.

Utilisez la notation **Grand O** pour décrire la complexité temporelle de cette fonction, qui est la suivante :

```
def chessboardSpace(numberOfGrains):
    chessboardSpaces = 1
    placedGrains = 1
    while placedGrains < numberOfGrains:
        placedGrains *= 2
        chessboardSpaces += 1
    return chessboardSpaces
```

Correction :

Cela correspond à **O(log N).** Dans ce cas, N représente le nombre **numberOfGrains**, qui est passé en paramètre à la fonction. La boucle s'exécute tant que **placedGrains**< **numberOfGrains**, mais **placedGrain** commence à 1 et double à chaque itération de la boucle. Par exemple, si **numberOfGrains** était de 256, on double **placedGrains** neuf fois jusqu'à atteindre 256, ce qui signifie que la boucle s'exécute neuf fois pour un N de 256. Si **numberOfGrains** était de 512, la boucle s'exécute dix fois, et si **numberOfGrains** était de 1024, la boucle s'exécute onze fois. Parce que la boucle s'exécute une fois de plus à chaque fois que N est doublé, cela correspond à **O(log N).**

Exercice 8:

Remplacez les points d'interrogation dans le tableau suivant pour décrire combien d'étapes se produisent pour un nombre donné d'éléments de données à travers divers types de notation **Grand O** :

N Elements	O(N)	O(log N)	O(N^2)
100	100	?	?
2000	?	?	?

Correction : Voici le tableau complet :

N Elements	O(N)	O(log N)	O(N^2)
100	100	Environ 7	10.000
2000	2000	Environ 11	4.000.000

Exercice9 : Si nous avons un algorithme en O(n^2) qui traite un tableau et que nous constatons qu'il prend 256 étapes, quelle est la taille du tableau ?

Correction : Le tableau aurait n = 16 éléments, car 16^2 équivaut à 256. C'est-à-dire que la racine carrée de 256 est égale à 16 ($256 = n^2 \Rightarrow n = \sqrt{256} = 16$)

Exercice10: Utilisez la notation **Grand O** pour décrire la complexité temporelle de la fonction suivante. Elle trouve le produit le plus élevé parmi toutes les paires de deux nombres dans un tableau donné :

```
def greatestProduct(array):
    greatestProductSoFar = array[0] * array[1]
    for i, iVal in enumerate(array):
        for j, jVal in enumerate(array):
            if i != j and iVal * jVal > greatestProductSoFar:
                greatestProductSoFar = iVal * jVal
    return greatestProductSoFar
```

Correction :

L'algorithme a une complexité temporelle de **O(n^2)**. n, dans ce cas, représente la taille du tableau. Nous avons une boucle externe qui itère sur le tableau n fois, et pour chacune de ces itérations, une boucle interne itère également sur le même tableau n fois. Cela entraîne n^2 opérations.

Exercice11 : La fonction suivante trouve le plus grand nombre unique dans un tableau, mais elle a une efficacité de **O(n^2).** Réécrivez la fonction pour qu'elle devienne plus rapide avec une efficacité de **O(n).**

```
def greatestNumber(array):
    for i in array:
        # Supposons pour le moment que i est le plus grand :
        isIValTheGreatest = True
        for j in array:
            # Si nous trouvons une autre valeur qui est plus grande que i,
            # i n'est pas le plus grand :
            if j > i:
                isIValTheGreatest = False
        # Si, à la fin de la vérification de tous les autres nombres, i
        # est toujours le plus grand, cela signifie qu'i est le plus grand nombre :
        if isIValTheGreatest:
            return i
```

Correction : La version suivante est en O(n), car nous n'itérons qu'une seule fois à travers le tableau :

```
def greatestNumber(array):
    greatestNumberSoFar = array[0]
```

```
    for i in array:
        if i > greatestNumberSoFar:
            greatestNumberSoFar = i
    return greatestNumberSoFar
```

Exercice12 : Utilisez la notation **Grand O** pour décrire la complexité temporelle d'un algorithme qui effectue 4n + 16 étapes.

Correction : Après avoir éliminé les constantes, nous pouvons réduire l'expression à **O(n).**

Exercice13 : Utilisez la notation **Grand O** pour décrire la complexité temporelle d'un algorithme qui effectue $2n^2$

Correction : Après avoir éliminé les constantes, nous pouvons réduire l'expression à **$O(n^2)$.**

Exercice 14:

Utilisez la notation **Grand O** pour décrire la complexité temporelle de la fonction suivante.

Explication : Cette fonction résout un problème célèbre connu sous le nom de "**recherche d'une aiguille dans une botte de foin**". Tant l'aiguille (needle الإبرة) que la botte de foin (haystack كومة قش) sont des chaînes de caractères. Par exemple, si l'aiguille est "def" et la botte de foin est "abcdefghi", l'aiguille se trouve quelque part dans la botte de foin, car "def" est une sous-chaîne de "abcdefghi". Cependant, si l'aiguille est "dd", elle ne peut pas être trouvée dans la botte de foin "abcdefghi". Cette fonction renvoie vrai ou faux, en fonction de la présence ou de l'absence de l'aiguille dans la botte de foin :

```
def find_needle(needle, haystack):
    needle_index = 0
    haystack_index = 0
    while haystack_index < len(haystack):
        if needle[needle_index] == haystack[haystack_index]:
            found_needle = True
            while needle_index < len(needle):
                if needle[needle_index] != haystack[haystack_index + needle_index]:
                    found_needle = False
                    break
                needle_index += 1
            if found_needle:
                return True
            needle_index = 0
        haystack_index += 1
    return False
```

Correction :

Hypothèse : le nombre de caractères dans l'aiguille = N et le nombre de caractères dans la botte de foin = M.

Dans le pire des cas, cet algorithme s'exécute en un nombre de fois égal au produit du nombre de caractères dans "l'aiguille" par le nombre de caractères dans la "botte de foin". Étant donné que l'aiguille et la botte de foin peuvent avoir des nombres de caractères différents, cela se traduit par une complexité en temps de O(N * M).